Lab Assignment 3: Transformer Model

Name	Krushna Prashant Warule	**Dept**	Computer
Roll No	_____	**Div**	B
Subject	Advance Data Science	**Assignment No**	3
GitHub	https://github.com/krushnawarule/Advance_Data_Science_Lab_SCOE/tree/main/Assignment_3_Transformer		

1. Aim

To design and implement a Transformer model for a sequence-based task (sentiment classification) where the attention mechanism identifies relationships between tokens and produces the appropriate output.

2. Theory

A Transformer replaces recurrence with **self-attention**. Each token is converted into Query, Key and Value vectors; attention weights = softmax(QK^T/sqrt(d)) decide how much each token should look at every other token, and the output is the weighted sum of the Values. Because attention has no notion of order, **positional encoding** (sine/cosine) is added to the embeddings. A **residual connection** and a **feed-forward network** complete the encoder block.

Model used: Embedding(16) + Positional Encoding -> Self-Attention -> Residual -> Feed-Forward(32) -> Residual -> Mean pooling -> Sigmoid. It is implemented from scratch in NumPy with a manually written backward pass; a numerical gradient check confirmed the gradients are correct (error about 1e-11).

3. Dataset

Simulated sentiment dataset: 800 sentences of 8 words each (34-word vocabulary) containing a positive or negative word; about 15% of sentences contain a negation (for example 'not good' becomes negative). Saved as `dataset.csv` (text, label). Split: 640 train, 160 test.

4. Code (core part; full code in code.py)

```
pos = np.arange(T)[:, None] / 10000 ** (2 * (np.arange(D)[None, :] // 2) / D)
PE = np.where(np.arange(D) % 2 == 0, np.sin(pos), np.cos(pos))       # positional encoding
r = lambda *s: np.random.randn(*s) * 0.2
P = {"E": r(len(vocab), D), "Wq": r(D, D), "Wk": r(D, D), "Wv": r(D, D),
     "W1": r(D, FF), "b1": np.zeros(FF), "W2": r(FF, D), "b2": np.zeros(D), "wc": r(D), "bc": np.zeros(1)}

def forward(tok):
    X = P["E"][tok] + PE                                              # embedding + position
    Q, K, V = X @ P["Wq"], X @ P["Wk"], X @ P["Wv"]
    S = Q @ K.T / np.sqrt(D)                                          # scaled dot-product attention
    A = np.exp(S - S.max(1, keepdims=True)); A /= A.sum(1, keepdims=True)
    H1 = X + A @ V                                                    # residual connection
    Z = np.maximum(H1 @ P["W1"] + P["b1"], 0)                         # feed-forward network
    H2 = H1 + Z @ P["W2"] + P["b2"]                                   # residual connection
    pooled = H2.mean(0)                                               # average over tokens
    prob = 1 / (1 + np.exp(-(pooled @ P["wc"] + P["bc"][0])))         # sigmoid classifier
    return prob, (tok, X, Q, K, V, A, H1, Z, pooled)

def backward(label, prob, c):
    tok, X, Q, K, V, A, H1, Z, pooled = c; g = {}
    dl = prob - label
    g["wc"] = pooled * dl; g["bc"] = np.array([dl]); dH2 = np.tile(P["wc"] * dl / T, (T, 1))
    g["W2"] = Z.T @ dH2; g["b2"] = dH2.sum(0)
    dpre = (dH2 @ P["W2"].T) * (Z > 0)
    g["W1"] = H1.T @ dpre; g["b1"] = dpre.sum(0); dH1 = dH2 + dpre @ P["W1"].T
    dA = dH1 @ V.T; dV = A.T @ dH1
    dS = A * (dA - (dA * A).sum(1, keepdims=True))
    dQ, dK = dS @ K / np.sqrt(D), dS.T @ Q / np.sqrt(D)
    g["Wq"], g["Wk"], g["Wv"] = X.T @ dQ, X.T @ dK, X.T @ dV
    dX = dH1 + dQ @ P["Wq"].T + dK @ P["Wk"].T + dV @ P["Wv"].T
    g["E"] = np.zeros_like(P["E"]); np.add.at(g["E"], tok, dX)
    return g
```

5. Output

Console output (output.txt):

```
Gradient check max error: 1.25e-11 (backprop is correct if this is tiny)
Vocabulary=34 words | sentences=800 | train=640 test=160
Trainable parameters: 2401
Epoch  1/30  train_acc=0.9453  test_acc=0.9125
Epoch  5/30  train_acc=0.9359  test_acc=0.9563
Epoch 10/30  train_acc=1.0000  test_acc=1.0000
```

```
Epoch 15/30  train_acc=1.0000  test_acc=1.0000
Epoch 20/30  train_acc=1.0000  test_acc=1.0000
Epoch 25/30  train_acc=1.0000  test_acc=1.0000
Epoch 30/30  train_acc=1.0000  test_acc=1.0000

Final Test Accuracy: 100.00%
'the movie was not good and really boring' -> Negative (0.01)
'this film was really amazing and the story' -> Positive (1.00)
'the plot was not bad but i felt it' -> Positive (1.00)
```

Output images:

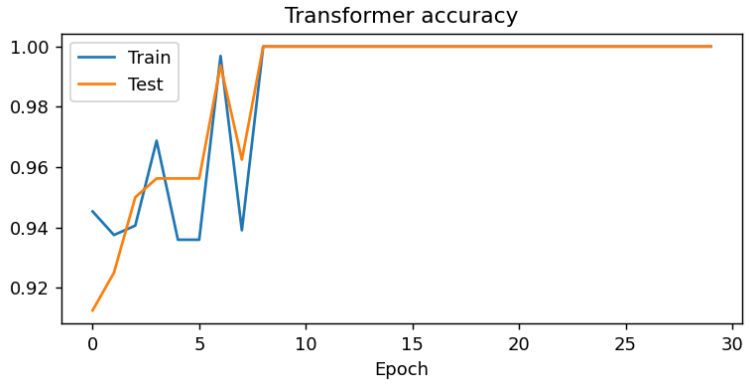


Fig 1: Training and test accuracy per epoch

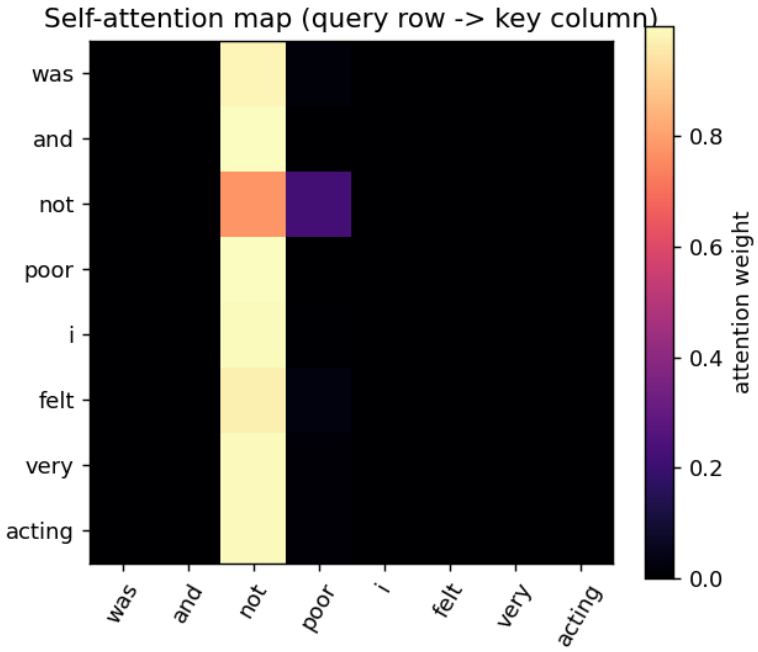


Fig 2: Self-attention heat-map for a sentence with a negation

6. Conclusion

The Transformer reached **100% test accuracy** on the simulated dataset and correctly handled negations such as 'not bad' and 'not good'. The attention heat-map shows that the tokens attend strongly to the negation word 'not', which explains how the model flips the sentiment. The perfect score is mainly because the simulated data is simple and the vocabulary is small, so it should not be read as performance on real reviews. For real data, multi-head attention, layer normalisation and several stacked encoder blocks would be used.

7. References

[1] A. Vaswani et al., 'Attention Is All You Need', NeurIPS 2017.

[2] J. Alammar, 'The Illustrated Transformer', jalammar.github.io.

[3] I. Goodfellow, Y. Bengio, A. Courville, Deep Learning, MIT Press, 2016.

[4] D. Kingma, J. Ba, 'Adam: A Method for Stochastic Optimization', ICLR 2015.